

Andrés Matías González Ros - Práctica Final Gráfica

Generado por Doxygen 1.13.2

1 Índice de clases	1
1.1 Lista de clases	1
2 Índice de archivos	3
2.1 Lista de archivos	3
3 Documentación de clases	5
3.1 Referencia de la clase <code>udit::Camera</code>	5
3.1.1 Descripción detallada	5
3.1.2 Documentación de constructores y destructores	5
3.1.2.1 <code>Camera()</code>	5
3.1.3 Documentación de funciones miembro	6
3.1.3.1 <code>get_view_matrix()</code>	6
3.1.3.2 <code>process_keyboard()</code>	6
3.1.3.3 <code>process_mouse_motion()</code>	6
3.1.3.4 <code>start_camera_control()</code>	7
3.2 Referencia de la clase <code>udit::Cone</code>	7
3.2.1 Descripción detallada	7
3.2.2 Documentación de constructores y destructores	7
3.2.2.1 <code>Cone()</code>	7
3.2.2.2 <code>~Cone()</code>	8
3.2.3 Documentación de funciones miembro	8
3.2.3.1 <code>render()</code>	8
3.3 Referencia de la clase <code>udit::Cylinder</code>	8
3.3.1 Descripción detallada	9
3.3.2 Documentación de constructores y destructores	9
3.3.2.1 <code>Cylinder()</code>	9
3.3.2.2 <code>~Cylinder()</code>	9
3.3.3 Documentación de funciones miembro	9
3.3.3.1 <code>render()</code>	9
3.4 Referencia de la clase <code>udit::Heightmap</code>	10
3.4.1 Descripción detallada	10
3.4.2 Documentación de constructores y destructores	10
3.4.2.1 <code>Heightmap()</code>	10
3.4.2.2 <code>~Heightmap()</code>	11
3.4.3 Documentación de funciones miembro	11
3.4.3.1 <code>getTextureID()</code>	11
3.4.3.2 <code>render()</code>	11
3.5 Referencia de la estructura <code>udit::Window::OpenGL_Context_Settings</code>	11
3.5.1 Descripción detallada	12
3.5.2 Documentación de datos miembro	12
3.5.2.1 <code>core_profile</code>	12
3.5.2.2 <code>depth_buffer_size</code>	12

3.5.2.3 enable_vsync	12
3.5.2.4 stencil_buffer_size	12
3.5.2.5 version_major	12
3.5.2.6 version_minor	12
3.6 Referencia de la estructura Window::OpenGL_Context_Settings	13
3.6.1 Descripción detallada	13
3.6.2 Documentación de datos miembro	13
3.6.2.1 core_profile	13
3.6.2.2 depth_buffer_size	13
3.6.2.3 enable_vsync	13
3.6.2.4 stencil_buffer_size	13
3.6.2.5 version_major	14
3.6.2.6 version_minor	14
3.7 Referencia de la clase udit::Plane	14
3.7.1 Descripción detallada	14
3.7.2 Documentación de constructores y destructores	14
3.7.2.1 Plane()	14
3.7.2.2 ~Plane()	15
3.7.3 Documentación de funciones miembro	15
3.7.3.1 render()	15
3.8 Referencia de la clase Scene	15
3.8.1 Descripción detallada	16
3.8.2 Documentación de constructores y destructores	16
3.8.2.1 Scene()	16
3.8.3 Documentación de funciones miembro	16
3.8.3.1 handle_mouse_motion()	16
3.8.3.2 render()	17
3.8.3.3 resize()	17
3.8.3.4 update()	17
3.9 Referencia de la clase udit::Scene	17
3.9.1 Descripción detallada	18
3.9.2 Documentación de constructores y destructores	18
3.9.2.1 Scene()	18
3.9.3 Documentación de funciones miembro	18
3.9.3.1 handle_mouse_motion()	18
3.9.3.2 render()	19
3.9.3.3 resize()	19
3.9.3.4 update()	19
3.10 Referencia de la clase udit::Skybox	20
3.10.1 Descripción detallada	20
3.10.2 Documentación de constructores y destructores	20
3.10.2.1 Skybox()	20

3.10.2.2 ~Skybox()	21
3.10.3 Documentación de funciones miembro	21
3.10.3.1 get_texture_id()	21
3.10.3.2 render()	21
3.10.3.3 set_texture()	21
3.11 Referencia de la clase udit::Sphere	22
3.11.1 Descripción detallada	22
3.11.2 Documentación de constructores y destructores	22
3.11.2.1 Sphere()	22
3.11.2.2 ~Sphere()	22
3.11.3 Documentación de funciones miembro	23
3.11.3.1 render()	23
3.12 Referencia de la clase udit::TextureLoader	23
3.12.1 Descripción detallada	23
3.12.2 Documentación de constructores y destructores	23
3.12.2.1 TextureLoader()	23
3.12.2.2 ~TextureLoader()	24
3.12.3 Documentación de funciones miembro	24
3.12.3.1 loadCubemap()	24
3.12.3.2 loadTexture()	24
3.13 Referencia de la clase udit::Window	25
3.13.1 Descripción detallada	25
3.13.2 Documentación de las enumeraciones miembro de la clase	25
3.13.2.1 Position	25
3.13.3 Documentación de constructores y destructores	26
3.13.3.1 Window() [1/4]	26
3.13.3.2 Window() [2/4]	26
3.13.3.3 ~Window()	27
3.13.3.4 Window() [3/4]	27
3.13.3.5 Window() [4/4]	27
3.13.4 Documentación de funciones miembro	27
3.13.4.1 operator=() [1/2]	27
3.13.4.2 operator=() [2/2]	27
3.13.4.3 swap_buffers()	28
3.14 Referencia de la clase Window	28
3.14.1 Descripción detallada	29
3.14.2 Documentación de las enumeraciones miembro de la clase	29
3.14.2.1 Position	29
3.14.3 Documentación de constructores y destructores	30
3.14.3.1 Window() [1/4]	30
3.14.3.2 Window() [2/4]	30
3.14.3.3 Window() [3/4]	31

3.14.3.4 Window() [4/4]	31
3.14.3.5 ~Window()	31
3.14.4 Documentación de funciones miembro	31
3.14.4.1 operator=() [1/2]	31
3.14.4.2 operator=() [2/2]	31
3.14.4.3 swap_buffers()	32
4 Documentación de archivos	33
4.1 Camera.hpp	33
4.2 Cone.hpp	34
4.3 Cylinder.hpp	34
4.4 Heightmap.hpp	35
4.5 Plane.hpp	35
4.6 Referencia del archivo Code/Headers/Scene.hpp	36
4.6.1 Descripción detallada	37
4.7 Scene.hpp	37
4.8 Referencia del archivo Code/Headers/Skybox.hpp	38
4.8.1 Descripción detallada	39
4.9 Skybox.hpp	39
4.10 Referencia del archivo Code/Headers/Sphere.hpp	39
4.10.1 Descripción detallada	40
4.11 Sphere.hpp	40
4.12 Referencia del archivo Code/Headers/TextureLoader.hpp	41
4.12.1 Descripción detallada	41
4.13 TextureLoader.hpp	41
4.14 Referencia del archivo Code/Headers/Window.hpp	42
4.14.1 Descripción detallada	42
4.15 Window.hpp	42
Índice alfabético	45

Capítulo 1

Índice de clases

1.1. Lista de clases

Lista de clases, estructuras, uniones e interfaces con breves descripciones:

udit::Camera	Clase que representa una cámara 3D en un espacio tridimensional	5
udit::Cone	Representa un cono 3D con un número de divisiones, radio y altura	7
udit::Cylinder	Clase que representa un cilindro en un espacio 3D	8
udit::Heightmap	Clase que representa un mapa de altura, utilizado para generar un terreno	10
udit::Window::OpenGL_Context_Settings	Estructura que contiene los detalles del contexto OpenGL	11
Window::OpenGL_Context_Settings	Estructura que contiene los detalles del contexto OpenGL	13
udit::Plane	Clase para representar un plano 3D	14
Scene	Clase para representar y gestionar una escena 3D	15
udit::Scene	Clase para representar y gestionar una escena 3D	17
udit::Skybox	Clase que representa un skybox en una escena 3D	20
udit::Sphere	Clase que representa una esfera 3D generada por subdivisiones esféricas	22
udit::TextureLoader	Clase para cargar texturas en OpenGL	23
udit::Window	Clase que gestiona la creación y manipulación de una ventana con contexto OpenGL	25
Window	Clase que gestiona la creación y manipulación de una ventana con contexto OpenGL	28

Capítulo 2

Índice de archivos

2.1. Lista de archivos

Lista de todos los archivos documentados y con breves descripciones:

Code/Headers/ Camera.hpp	33
Code/Headers/ Cone.hpp	34
Code/Headers/ Cylinder.hpp	34
Code/Headers/ Heightmap.hpp	35
Code/Headers/ Plane.hpp	35
Code/Headers/ Scene.hpp	
Definición de la clase Scene , encargada de manejar y renderizar la escena 3D	36
Code/Headers/ Skybox.hpp	38
Code/Headers/ Sphere.hpp	39
Code/Headers/ TextureLoader.hpp	41
Code/Headers/ Window.hpp	42

Capítulo 3

Documentación de clases

3.1. Referencia de la clase `udit::Camera`

Clase que representa una cámara 3D en un espacio tridimensional.

```
#include <Camera.hpp>
```

Métodos públicos

- `Camera` (`glm::vec3 start_position`, `glm::vec3 start_up`, `float start_yaw`, `float start_pitch`)
Constructor de la clase `Camera`.
- `glm::mat4 get_view_matrix` () const
Obtiene la matriz de vista de la cámara.
- void `process_keyboard` (const `UInt8 *state`, `float delta_time`)
Procesa el movimiento del teclado para mover la cámara.
- void `start_camera_control` ()
Inicia el control de la cámara mediante entradas de ratón.
- void `process_mouse_motion` (`float xrel`, `float yrel`)
Procesa el movimiento del ratón para rotar la cámara.

3.1.1. Descripción detallada

Clase que representa una cámara 3D en un espacio tridimensional.

La clase permite controlar la posición y orientación de una cámara, manejar el movimiento por teclado y la rotación mediante el ratón. Proporciona métodos para obtener la matriz de vista y actualizar los vectores de la cámara.

3.1.2. Documentación de constructores y destructores

3.1.2.1. `Camera()`

```
udit::Camera::Camera (  
    glm::vec3 start_position,  
    glm::vec3 start_up,  
    float start_yaw,  
    float start_pitch)
```

Constructor de la clase `Camera`.

Inicializa la cámara con una posición, una orientación inicial y los valores de yaw y pitch.

Parámetros

<i>start_position</i>	Posición inicial de la cámara.
<i>start_up</i>	Vector que define la dirección hacia arriba de la cámara.
<i>start_yaw</i>	Ángulo de orientación inicial alrededor del eje Y.
<i>start_pitch</i>	Ángulo de orientación inicial alrededor del eje X.

3.1.3. Documentación de funciones miembro**3.1.3.1. `get_view_matrix()`**

```
glm::mat4 udit::Camera::get_view_matrix () const
```

Obtiene la matriz de vista de la cámara.

La matriz de vista es útil para la proyección de la cámara en el espacio tridimensional.

Devuelve

glm::mat4 La matriz de vista.

3.1.3.2. `process_keyboard()`

```
void udit::Camera::process_keyboard (
    const Uint8 * state,
    float delta_time)
```

Procesa el movimiento del teclado para mover la cámara.

Este método ajusta la posición de la cámara basándose en las teclas presionadas durante un intervalo de tiempo específico.

Parámetros

<i>state</i>	Puntero a un array que representa el estado de las teclas.
<i>delta_time</i>	Tiempo transcurrido entre fotogramas.

3.1.3.3. `process_mouse_motion()`

```
void udit::Camera::process_mouse_motion (
    float xrel,
    float yrel)
```

Procesa el movimiento del ratón para rotar la cámara.

Este método ajusta los ángulos de orientación de la cámara según el movimiento del ratón.

Parámetros

<i>xrel</i>	Desplazamiento horizontal del ratón.
<i>yrel</i>	Desplazamiento vertical del ratón.

3.1.3.4. start_camera_control()

```
void udit::Camera::start_camera_control ()
```

Inicia el control de la cámara mediante entradas de ratón.

Este método configura los parámetros necesarios para controlar la cámara usando el ratón.

La documentación de esta clase está generada de los siguientes archivos:

- Code/Headers/Camera.hpp
- Code/Sources/Camera.cpp

3.2. Referencia de la clase udit::Cone

Representa un cono 3D con un número de divisiones, radio y altura.

```
#include <Cone.hpp>
```

Métodos públicos

- [Cone](#) (int d, GLfloat r, GLfloat h)
Constructor de la clase [Cone](#).
- [~Cone](#) ()
Destructor de la clase [Cone](#).
- void [render](#) ()
Renderiza el cono en la escena.

3.2.1. Descripción detallada

Representa un cono 3D con un número de divisiones, radio y altura.

La clase permite la creación de un cono 3D, la gestión de sus geometrías (coordenadas, índices y coordenadas de textura) y el renderizado del mismo utilizando OpenGL. El cono se genera a partir de las especificaciones de radio, altura y divisiones.

3.2.2. Documentación de constructores y destructores

3.2.2.1. Cone()

```
udit::Cone::Cone (
    int d,
    GLfloat r,
    GLfloat h)
```

Constructor de la clase [Cone](#).

Inicializa el cono con el número de divisiones, el radio y la altura especificados. Este constructor también prepara las estructuras necesarias para el renderizado.

Parámetros

d	Número de divisiones del cono.
r	Radio de la base del cono.
h	Altura del cono.

3.2.2.2. `~Cone()`

```
udit::Cone::~~Cone ()
```

Destructor de la clase [Cone](#).

Libera los recursos utilizados por los buffers de OpenGL (VAO y VBOs).

3.2.3. Documentación de funciones miembro**3.2.3.1. `render()`**

```
void udit::Cone::render ()
```

Renderiza el cono en la escena.

Este método utiliza OpenGL para dibujar el cono en la escena utilizando los VBOs y VAO previamente generados.

La documentación de esta clase está generada de los siguientes archivos:

- Code/Headers/Cone.hpp
- Code/Sources/Cone.cpp

3.3. Referencia de la clase `udit::Cylinder`

Clase que representa un cilindro en un espacio 3D.

```
#include <Cylinder.hpp>
```

Métodos públicos

- [Cylinder](#) (int stack, int slice, GLfloat r, GLfloat h)
Constructor de la clase [Cylinder](#).
- [~Cylinder](#) ()
Destructor de la clase [Cylinder](#).
- void [render](#) ()
Renderiza el cilindro en la pantalla.

3.3.1. Descripción detallada

Clase que representa un cilindro en un espacio 3D.

La clase se encarga de generar la geometría de un cilindro, con sus vértices, coordenadas de textura e índices, y de renderizarlo utilizando OpenGL.

3.3.2. Documentación de constructores y destructores

3.3.2.1. `Cylinder()`

```
udit::Cylinder::Cylinder (
    int stack,
    int slice,
    GLfloat r,
    GLfloat h)
```

Constructor de la clase [Cylinder](#).

Este constructor inicializa los parámetros del cilindro, como la cantidad de pilas (stacks), la cantidad de segmentos (slices), el radio y la altura, y luego genera la geometría asociada a esos parámetros.

Parámetros

<i>stack</i>	Número de divisiones verticales del cilindro (pilas).
<i>slice</i>	Número de divisiones alrededor del eje del cilindro (segmentos).
<i>r</i>	Radio de la base del cilindro.
<i>h</i>	Altura del cilindro.

3.3.2.2. `~Cylinder()`

```
udit::Cylinder::~~Cylinder ()
```

Destructor de la clase [Cylinder](#).

Este destructor se encarga de liberar los recursos utilizados por el cilindro, como los buffers de OpenGL.

3.3.3. Documentación de funciones miembro

3.3.3.1. `render()`

```
void udit::Cylinder::render ()
```

Renderiza el cilindro en la pantalla.

Este método utiliza los buffers de OpenGL para dibujar el cilindro en la pantalla.

La documentación de esta clase está generada de los siguientes archivos:

- Code/Headers/Cylinder.hpp
- Code/Sources/Cylinder.cpp

3.4. Referencia de la clase `udit::Heightmap`

Clase que representa un mapa de altura, utilizado para generar un terreno.

```
#include <Heightmap.hpp>
```

Métodos públicos

- [Heightmap](#) (const std::string &heightmapPath, float width, float height, float maxHeight)
Constructor de la clase [Heightmap](#).
- [~Heightmap](#) ()
Destructor de la clase [Heightmap](#).
- void [render](#) () const
Renderiza el mapa de altura en la pantalla.
- GLuint [getTextureID](#) () const
Obtiene el ID de la textura del mapa de altura.

3.4.1. Descripción detallada

Clase que representa un mapa de altura, utilizado para generar un terreno.

Esta clase maneja la carga de un mapa de altura, la generación de la malla correspondiente en función de los datos de altura, y la renderización del terreno usando OpenGL. El mapa de altura es una imagen de escala de grises que se utiliza para determinar las elevaciones del terreno en cada punto de la malla.

3.4.2. Documentación de constructores y destructores

3.4.2.1. `Heightmap()`

```
udit::Heightmap::Heightmap (
    const std::string & heightmapPath,
    float width,
    float height,
    float maxHeight)
```

Constructor de la clase [Heightmap](#).

Este constructor carga los datos del mapa de altura desde un archivo de imagen, genera la malla de vértices basada en esos datos y carga la textura del mapa de altura.

Parámetros

<i>heightmapPath</i>	Ruta del archivo de imagen que contiene los datos del mapa de altura.
<i>width</i>	Ancho del terreno en unidades del mundo.
<i>height</i>	Altura del terreno en unidades del mundo.
<i>maxHeight</i>	Altura máxima en el mapa de altura.

3.4.2.2. `~Heightmap()`

```
udit::Heightmap::~Heightmap ()
```

Destructor de la clase [Heightmap](#).

Este destructor se encarga de liberar los recursos asociados con la textura y los buffers de OpenGL.

3.4.3. Documentación de funciones miembro

3.4.3.1. `getTextureID()`

```
GLuint udit::Heightmap::getTextureID () const [inline]
```

Obtiene el ID de la textura del mapa de altura.

Este método devuelve el identificador de la textura de OpenGL utilizada para el mapa de altura.

Devuelve

GLuint El identificador de la textura.

3.4.3.2. `render()`

```
void udit::Heightmap::render () const
```

Renderiza el mapa de altura en la pantalla.

Este método utiliza los buffers de OpenGL para dibujar el mapa de altura en la pantalla.

La documentación de esta clase está generada de los siguientes archivos:

- Code/Headers/Heightmap.hpp
- Code/Sources/Heightmap.cpp

3.5. Referencia de la estructura `udit::Window::OpenGL_Context_Settings`

Estructura que contiene los detalles del contexto OpenGL.

```
#include <Window.hpp>
```

Atributos públicos

- unsigned [version_major](#) = 3
- unsigned [version_minor](#) = 3
- bool [core_profile](#) = true
- unsigned [depth_buffer_size](#) = 24
- unsigned [stencil_buffer_size](#) = 0
- bool [enable_vsync](#) = true

3.5.1. Descripción detallada

Estructura que contiene los detalles del contexto OpenGL.

Esta estructura permite configurar los detalles del contexto de OpenGL al crear la ventana.

3.5.2. Documentación de datos miembro

3.5.2.1. core_profile

```
bool udit::Window::OpenGL_Context_Settings::core_profile = true
```

Si se debe usar el perfil core de OpenGL.

3.5.2.2. depth_buffer_size

```
unsigned udit::Window::OpenGL_Context_Settings::depth_buffer_size = 24
```

Tamaño del buffer de profundidad.

3.5.2.3. enable_vsync

```
bool udit::Window::OpenGL_Context_Settings::enable_vsync = true
```

Si se debe habilitar el V-Sync.

3.5.2.4. stencil_buffer_size

```
unsigned udit::Window::OpenGL_Context_Settings::stencil_buffer_size = 0
```

Tamaño del buffer de stencil.

3.5.2.5. version_major

```
unsigned udit::Window::OpenGL_Context_Settings::version_major = 3
```

Versión principal de OpenGL.

3.5.2.6. version_minor

```
unsigned udit::Window::OpenGL_Context_Settings::version_minor = 3
```

Versión secundaria de OpenGL.

La documentación de esta estructura está generada del siguiente archivo:

- Code/Headers/[Window.hpp](#)

3.6. Referencia de la estructura Window::OpenGL_Context_Settings

Estructura que contiene los detalles del contexto OpenGL.

```
#include <Window.hpp>
```

Atributos públicos

- unsigned `version_major` = 3
- unsigned `version_minor` = 3
- bool `core_profile` = true
- unsigned `depth_buffer_size` = 24
- unsigned `stencil_buffer_size` = 0
- bool `enable_vsync` = true

3.6.1. Descripción detallada

Estructura que contiene los detalles del contexto OpenGL.

Esta estructura permite configurar los detalles del contexto de OpenGL al crear la ventana.

3.6.2. Documentación de datos miembro

3.6.2.1. `core_profile`

```
bool udit::Window::OpenGL_Context_Settings::core_profile = true
```

Si se debe usar el perfil core de OpenGL.

3.6.2.2. `depth_buffer_size`

```
unsigned udit::Window::OpenGL_Context_Settings::depth_buffer_size = 24
```

Tamaño del buffer de profundidad.

3.6.2.3. `enable_vsync`

```
bool udit::Window::OpenGL_Context_Settings::enable_vsync = true
```

Si se debe habilitar el V-Sync.

3.6.2.4. `stencil_buffer_size`

```
unsigned udit::Window::OpenGL_Context_Settings::stencil_buffer_size = 0
```

Tamaño del buffer de stencil.

3.6.2.5. version_major

```
unsigned udit::Window::OpenGL_Context_Settings::version_major = 3
```

Versión principal de OpenGL.

3.6.2.6. version_minor

```
unsigned udit::Window::OpenGL_Context_Settings::version_minor = 3
```

Versión secundaria de OpenGL.

La documentación de esta estructura está generada del siguiente archivo:

- Code/Headers/[Window.hpp](#)

3.7. Referencia de la clase udit::Plane

Clase para representar un plano 3D.

```
#include <Plane.hpp>
```

Métodos públicos

- [Plane](#) (int width, int height)
Constructor de la clase [Plane](#).
- [~Plane](#) ()
Destructor de la clase [Plane](#).
- void [render](#) ()
Renderiza el plano.

3.7.1. Descripción detallada

Clase para representar un plano 3D.

Esta clase genera un plano de malla utilizando OpenGL. El plano tiene una cuadrícula definida por su ancho y alto, y se utiliza para representar superficies planas.

3.7.2. Documentación de constructores y destructores

3.7.2.1. Plane()

```
udit::Plane::Plane (  
    int width,  
    int height)
```

Constructor de la clase [Plane](#).

Inicializa un plano con la cuadrícula definida por el ancho y alto proporcionados. Genera la geometría del plano (vértices, coordenadas de textura e índices).

Parámetros

<i>width</i>	El ancho del plano, define la cantidad de columnas de la cuadrícula.
<i>height</i>	La altura del plano, define la cantidad de filas de la cuadrícula.

3.7.2.2. ~Plane()

```
udit::Plane::~~Plane ()
```

Destructor de la clase [Plane](#).

Libera los recursos de OpenGL utilizados para almacenar la geometría del plano.

3.7.3. Documentación de funciones miembro**3.7.3.1. render()**

```
void udit::Plane::render ()
```

Renderiza el plano.

Dibuja el plano utilizando los datos de geometría generados con OpenGL.

La documentación de esta clase está generada de los siguientes archivos:

- Code/Headers/Plane.hpp
- Code/Sources/Plane.cpp

3.8. Referencia de la clase Scene

Clase para representar y gestionar una escena 3D.

```
#include <Scene.hpp>
```

Métodos públicos

- [Scene](#) (unsigned width, unsigned height)
Constructor de la clase [Scene](#).
- void [update](#) (float delta_time)
Actualiza la escena.
- void [render](#) ()
Renderiza la escena.
- void [resize](#) (unsigned width, unsigned height)
Cambia el tamaño de la ventana de la escena.
- void [handle_mouse_motion](#) (float xrel, float yrel)
Maneja el movimiento del ratón en la ventana.

3.8.1. Descripción detallada

Clase para representar y gestionar una escena 3D.

Esta clase encapsula la creación de objetos 3D (como esferas, conos, cilindros, planos, etc.), la configuración de la cámara, la carga de texturas, y la administración de los shaders para la representación visual de la escena. Permite renderizar la escena y actualizarla durante la ejecución del programa.

Nota

Esta clase ha sido modificada respecto a su versión original para incluir:

- Un entorno 3D con un skybox.
- La implementación de una cámara interactiva.
- Nuevos objetos como un cilindro, un cono, una esfera, un plano y un heightmap.
- Un cargador de texturas personalizado para los objetos 3D.

3.8.2. Documentación de constructores y destructores

3.8.2.1. Scene()

```
udit::Scene::Scene (
    unsigned width,
    unsigned height)
```

Constructor de la clase [Scene](#).

Inicializa la escena con los objetos 3D, la cámara, las texturas y los shaders necesarios para renderizar la escena correctamente.

Parámetros

<i>width</i>	Ancho de la ventana de la escena.
<i>height</i>	Alto de la ventana de la escena.

3.8.3. Documentación de funciones miembro

3.8.3.1. handle_mouse_motion()

```
void udit::Scene::handle_mouse_motion (
    float xrel,
    float yrel)
```

Maneja el movimiento del ratón en la ventana.

Permite controlar la cámara con el ratón, ajustando su orientación.

Parámetros

<i>xrel</i>	Movimiento del ratón en el eje X.
<i>yrel</i>	Movimiento del ratón en el eje Y.

3.8.3.2. `render()`

```
void udit::Scene::render ()
```

Renderiza la escena.

Este método dibuja todos los objetos 3D de la escena utilizando OpenGL, aplicando las transformaciones y shaders correspondientes.

3.8.3.3. `resize()`

```
void udit::Scene::resize (
    unsigned width,
    unsigned height)
```

Cambia el tamaño de la ventana de la escena.

Ajusta la relación de aspecto y las matrices de proyección en función del nuevo tamaño de la ventana para asegurar una correcta visualización de la escena.

Parámetros

<i>width</i>	Nuevo ancho de la ventana.
<i>height</i>	Nuevo alto de la ventana.

3.8.3.4. `update()`

```
void udit::Scene::update (
    float delta_time)
```

Actualiza la escena.

Este método actualiza los elementos de la escena en función del tiempo transcurrido, incluyendo las animaciones de los objetos y los movimientos de la cámara.

Parámetros

<i>delta_time</i>	Tiempo transcurrido desde la última actualización.
-------------------	--

La documentación de esta clase está generada de los siguientes archivos:

- Code/Headers/[Scene.hpp](#)
- Code/Sources/Scene.cpp

3.9. Referencia de la clase `udit::Scene`

Clase para representar y gestionar una escena 3D.

```
#include <Scene.hpp>
```

Métodos públicos

- **Scene** (unsigned width, unsigned height)
*Constructor de la clase **Scene**.*
- void **update** (float delta_time)
Actualiza la escena.
- void **render** ()
Renderiza la escena.
- void **resize** (unsigned width, unsigned height)
Cambia el tamaño de la ventana de la escena.
- void **handle_mouse_motion** (float xrel, float yrel)
Maneja el movimiento del ratón en la ventana.

3.9.1. Descripción detallada

Clase para representar y gestionar una escena 3D.

Esta clase encapsula la creación de objetos 3D (como esferas, conos, cilindros, planos, etc.), la configuración de la cámara, la carga de texturas, y la administración de los shaders para la representación visual de la escena. Permite renderizar la escena y actualizarla durante la ejecución del programa.

Nota

Esta clase ha sido modificada respecto a su versión original para incluir:

- Un entorno 3D con un skybox.
- La implementación de una cámara interactiva.
- Nuevos objetos como un cilindro, un cono, una esfera, un plano y un heightmap.
- Un cargador de texturas personalizado para los objetos 3D.

3.9.2. Documentación de constructores y destructores

3.9.2.1. Scene()

```
udit::Scene::Scene (
    unsigned width,
    unsigned height)
```

Constructor de la clase **Scene**.

Inicializa la escena con los objetos 3D, la cámara, las texturas y los shaders necesarios para renderizar la escena correctamente.

Parámetros

<i>width</i>	Ancho de la ventana de la escena.
<i>height</i>	Alto de la ventana de la escena.

3.9.3. Documentación de funciones miembro

3.9.3.1. handle_mouse_motion()

```
void udit::Scene::handle_mouse_motion (
    float xrel,
    float yrel)
```

Maneja el movimiento del ratón en la ventana.

Permite controlar la cámara con el ratón, ajustando su orientación.

Parámetros

<i>xrel</i>	Movimiento del ratón en el eje X.
<i>yrel</i>	Movimiento del ratón en el eje Y.

3.9.3.2. `render()`

```
void udit::Scene::render ()
```

Renderiza la escena.

Este método dibuja todos los objetos 3D de la escena utilizando OpenGL, aplicando las transformaciones y shaders correspondientes.

3.9.3.3. `resize()`

```
void udit::Scene::resize (  
    unsigned width,  
    unsigned height)
```

Cambia el tamaño de la ventana de la escena.

Ajusta la relación de aspecto y las matrices de proyección en función del nuevo tamaño de la ventana para asegurar una correcta visualización de la escena.

Parámetros

<i>width</i>	Nuevo ancho de la ventana.
<i>height</i>	Nuevo alto de la ventana.

3.9.3.4. `update()`

```
void udit::Scene::update (  
    float delta_time)
```

Actualiza la escena.

Este método actualiza los elementos de la escena en función del tiempo transcurrido, incluyendo las animaciones de los objetos y los movimientos de la cámara.

Parámetros

<i>delta_time</i>	Tiempo transcurrido desde la última actualización.
-------------------	--

La documentación de esta clase está generada de los siguientes archivos:

- [Code/Headers/Scene.hpp](#)
- [Code/Sources/Scene.cpp](#)

3.10. Referencia de la clase udit::Skybox

Clase que representa un skybox en una escena 3D.

```
#include <Skybox.hpp>
```

Métodos públicos

- [Skybox](#) (const std::vector< std::string > &faces)
Constructor de la clase [Skybox](#).
- [~Skybox](#) ()
Destructor de la clase [Skybox](#).
- void [set_texture](#) (GLuint texture_id)
Establece la textura del skybox.
- GLuint [get_texture_id](#) ()
Obtiene el identificador de la textura del skybox.
- void [render](#) ()
Renderiza el skybox en la escena.

3.10.1. Descripción detallada

Clase que representa un skybox en una escena 3D.

La clase [Skybox](#) se encarga de la creación y renderizado de un cubo que representa el fondo de la escena. Utiliza un conjunto de texturas (una para cada cara del cubo) y las mapea sobre un cubo 3D que se renderiza como fondo estático. También maneja la carga de texturas, la creación de buffers y la configuración de las propiedades necesarias para su renderizado en OpenGL.

3.10.2. Documentación de constructores y destructores

3.10.2.1. Skybox()

```
udit::Skybox::Skybox (
    const std::vector< std::string > & faces)
```

Constructor de la clase [Skybox](#).

Este constructor recibe un vector de cadenas de texto que contiene las rutas de las imágenes que se usarán para las seis caras del cubo del skybox.

Parámetros

<i>faces</i>	Rutas de las texturas para las caras del cubo.
--------------	--

3.10.2.2. ~Skybox()

```
udit::Skybox::~~Skybox ()
```

Destructor de la clase [Skybox](#).

Limpia los recursos asociados al skybox, incluyendo la liberación de la memoria usada para las texturas y los buffers.

3.10.3. Documentación de funciones miembro

3.10.3.1. get_texture_id()

```
GLuint udit::Skybox::get_texture_id ()
```

Obtiene el identificador de la textura del skybox.

Este método retorna el identificador de la textura que se usa para el skybox.

Devuelve

GLuint El identificador de la textura del skybox.

3.10.3.2. render()

```
void udit::Skybox::render ()
```

Renderiza el skybox en la escena.

Este método realiza el renderizado del cubo del skybox, usando la textura proporcionada para las seis caras del cubo. Debe ser llamado en cada ciclo de renderizado.

3.10.3.3. set_texture()

```
void udit::Skybox::set_texture (
    GLuint texture_id)
```

Establece la textura del skybox.

Este método recibe un identificador de textura y lo asigna al skybox.

Parámetros

<i>texture_id</i>	El identificador de la textura para el skybox.
-------------------	--

La documentación de esta clase está generada de los siguientes archivos:

- Code/Headers/[Skybox.hpp](#)
- Code/Sources/[Skybox.cpp](#)

3.11. Referencia de la clase udit::Sphere

Clase que representa una esfera 3D generada por subdivisiones esféricas.

```
#include <Sphere.hpp>
```

Métodos públicos

- [Sphere](#) (int latDivisions, int longDivisions, GLfloat radius)
Constructor de la clase [Sphere](#).
- [~Sphere](#) ()
Destructor de la clase [Sphere](#).
- void [render](#) ()
Renderiza la esfera en la escena.

3.11.1. Descripción detallada

Clase que representa una esfera 3D generada por subdivisiones esféricas.

La esfera es una representación 3D que se genera mediante un proceso de subdivisión en latitudes y longitudes, lo que da como resultado un conjunto de vértices, coordenadas de textura y un índice para representar la geometría de la esfera. Esta clase permite renderizar la esfera utilizando OpenGL.

3.11.2. Documentación de constructores y destructores

3.11.2.1. Sphere()

```
udit::Sphere::Sphere (
    int latDivisions,
    int longDivisions,
    GLfloat radius)
```

Constructor de la clase [Sphere](#).

Este constructor recibe las divisiones latitudinales y longitudinales, así como el radio de la esfera. Utiliza estos parámetros para generar la geometría de la esfera.

Parámetros

<i>latDivisions</i>	Número de divisiones latitudinales (longitud de la esfera).
<i>longDivisions</i>	Número de divisiones longitudinales (circunferencia de la esfera).
<i>radius</i>	Radio de la esfera.

3.11.2.2. ~Sphere()

```
udit::Sphere::~~Sphere ()
```

Destructor de la clase [Sphere](#).

El destructor elimina los recursos de OpenGL utilizados para la esfera, como los buffers de vértices, índices y coordenadas de textura.

3.11.3. Documentación de funciones miembro

3.11.3.1. render()

```
void udit::Sphere::render ()
```

Renderiza la esfera en la escena.

Este método dibuja la esfera utilizando los datos generados (vértices, índices y coordenadas de textura) y los buffers correspondientes.

La documentación de esta clase está generada de los siguientes archivos:

- Code/Headers/[Sphere.hpp](#)
- Code/Sources/[Sphere.cpp](#)

3.12. Referencia de la clase udit::TextureLoader

Clase para cargar texturas en OpenGL.

```
#include <TextureLoader.hpp>
```

Métodos públicos

- [TextureLoader](#) ()
Constructor de la clase [TextureLoader](#).
- [~TextureLoader](#) ()
Destructor de la clase [TextureLoader](#).
- GLuint [loadTexture](#) (const string &filePath)
Carga una textura 2D desde un archivo.
- GLuint [loadCubemap](#) (const vector< string > &faces)
Carga un cubemap desde una lista de archivos de imagen.

3.12.1. Descripción detallada

Clase para cargar texturas en OpenGL.

La clase [TextureLoader](#) maneja la carga de texturas 2D y cubemaps, proporcionando métodos para cargar imágenes desde archivos y generar las texturas correspondientes en OpenGL.

3.12.2. Documentación de constructores y destructores

3.12.2.1. TextureLoader()

```
udit::TextureLoader::TextureLoader ()
```

Constructor de la clase [TextureLoader](#).

Este constructor inicializa la clase. Actualmente no se utiliza para realizar ninguna operación en particular.

3.12.2.2. `~TextureLoader()`

```
udit::TextureLoader::~TextureLoader ()
```

Destructor de la clase [TextureLoader](#).

El destructor limpia cualquier recurso utilizado por la clase (si es necesario).

3.12.3. Documentación de funciones miembro

3.12.3.1. `loadCubemap()`

```
GLuint udit::TextureLoader::loadCubemap (  
    const vector< string > & faces)
```

Carga un cubemap desde una lista de archivos de imagen.

Este método carga una serie de imágenes que representan las caras de un cubemap.

Parámetros

<i>faces</i>	Lista de rutas de los archivos de imagen que representan las caras del cubemap.
--------------	---

Devuelve

GLuint ID del cubemap cargado.

3.12.3.2. `loadTexture()`

```
GLuint udit::TextureLoader::loadTexture (  
    const string & filePath)
```

Carga una textura 2D desde un archivo.

Este método carga una imagen desde un archivo y crea una textura 2D en OpenGL.

Parámetros

<i>filePath</i>	Ruta del archivo de imagen.
-----------------	-----------------------------

Devuelve

GLuint ID de la textura cargada.

La documentación de esta clase está generada de los siguientes archivos:

- Code/Headers/[TextureLoader.hpp](#)
- Code/Sources/[TextureLoader.cpp](#)

3.13. Referencia de la clase udit::Window

Clase que gestiona la creación y manipulación de una ventana con contexto OpenGL.

```
#include <Window.hpp>
```

Clases

- struct [OpenGL_Context_Settings](#)
Estructura que contiene los detalles del contexto OpenGL.

Tipos públicos

- enum [Position](#) { [UNDEFINED](#) = SDL_WINDOWPOS_UNDEFINED , [CENTERED](#) = SDL_WINDOWPOS_↵
CENTERED }
- Define las posiciones predefinidas para la ventana.*

Métodos públicos

- [Window](#) (const std::string &title, int left_x, int top_y, unsigned width, unsigned height, const [OpenGL_Context_Settings](#) &context_details)
Constructor de la clase [Window](#).
- [Window](#) (const char *title, int left_x, int top_y, unsigned width, unsigned height, const [OpenGL_Context_Settings](#) &context_details)
Constructor de la clase [Window](#) (versión con título como `const char*`).
- [~Window](#) ()
Destructor de la clase [Window](#).
- [Window](#) (const [Window](#) &)=delete
Constructor de copia deshabilitado.
- [Window](#) & [operator=](#) (const [Window](#) &)=delete
Operador de asignación deshabilitado.
- [Window](#) ([Window](#) &&other) noexcept
Constructor de movimiento.
- [Window](#) & [operator=](#) ([Window](#) &&other) noexcept
Operador de asignación por movimiento.
- void [swap_buffers](#) ()
Intercambia los buffers de la ventana.

3.13.1. Descripción detallada

Clase que gestiona la creación y manipulación de una ventana con contexto OpenGL.

Esta clase proporciona una interfaz sencilla para crear una ventana que utilice OpenGL como contexto de renderizado, configurando los detalles del contexto de OpenGL y ofreciendo funcionalidades como el intercambio de buffers.

3.13.2. Documentación de las enumeraciones miembro de la clase

3.13.2.1. Position

```
enum udit::Window::Position
```

Define las posiciones predefinidas para la ventana.

Esta enumeración contiene las posibles posiciones que la ventana puede tener en la pantalla.

Valores de enumeraciones

UNDEFINED	La posición de la ventana no está definida.
CENTERED	La ventana se centrará en la pantalla.

3.13.3. Documentación de constructores y destructores

3.13.3.1. Window() [1/4]

```
udit::Window::Window (
    const std::string & title,
    int left_x,
    int top_y,
    unsigned width,
    unsigned height,
    const OpenGL_Context_Settings & context_details) [inline]
```

Constructor de la clase [Window](#).

Este constructor crea una ventana con las especificaciones dadas y configura el contexto OpenGL utilizando los parámetros proporcionados.

Parámetros

<i>title</i>	Título de la ventana.
<i>left_x</i>	Posición en el eje X de la ventana.
<i>top_y</i>	Posición en el eje Y de la ventana.
<i>width</i>	Ancho de la ventana.
<i>height</i>	Alto de la ventana.
<i>context_details</i>	Detalles del contexto OpenGL.

3.13.3.2. Window() [2/4]

```
udit::Window::Window (
    const char * title,
    int left_x,
    int top_y,
    unsigned width,
    unsigned height,
    const OpenGL_Context_Settings & context_details)
```

Constructor de la clase [Window](#) (versión con título como `const char*`).

Este constructor permite crear una ventana con un título de tipo `const char*` y las especificaciones necesarias para el contexto OpenGL.

Parámetros

<i>title</i>	Título de la ventana.
<i>left_x</i>	Posición en el eje X de la ventana.
<i>top_y</i>	Posición en el eje Y de la ventana.
<i>width</i>	Ancho de la ventana.
<i>height</i>	Alto de la ventana.
<i>context_details</i>	Detalles del contexto OpenGL.

3.13.3.3. ~Window()

```
udit::Window::~~Window ()
```

Destructor de la clase [Window](#).

Este destructor destruye la ventana y limpia el contexto OpenGL asociado.

3.13.3.4. Window() [3/4]

```
udit::Window::Window (
    const Window & ) [delete]
```

Constructor de copia deshabilitado.

Este constructor de copia ha sido deshabilitado para evitar la copia del objeto [Window](#), ya que no tiene sentido clonar una ventana.

3.13.3.5. Window() [4/4]

```
udit::Window::Window (
    Window && other) [inline], [noexcept]
```

Constructor de movimiento.

Este constructor transfiere la propiedad de la ventana y el contexto OpenGL de un objeto [Window](#) a otro mediante el uso de `std::exchange`.

Parámetros

<i>other</i>	Objeto Window cuyo contenido se moverá.
--------------	---

3.13.4. Documentación de funciones miembro**3.13.4.1. operator=() [1/2]**

```
Window & udit::Window::operator= (
    const Window & ) [delete]
```

Operador de asignación deshabilitado.

El operador de asignación ha sido deshabilitado para evitar la asignación de un objeto [Window](#) a otro, ya que este tipo de operación no es válida en este contexto.

3.13.4.2. operator=() [2/2]

```
Window & udit::Window::operator= (
    Window && other) [inline], [noexcept]
```

Operador de asignación por movimiento.

Este operador transfiere la propiedad de la ventana y el contexto OpenGL de un objeto [Window](#) a otro mediante el uso de `std::exchange`.

Parámetros

<i>other</i>	Objeto Window cuyo contenido se moverá.
--------------	---

Devuelve

Referencia al objeto [Window](#) actualizado.

3.13.4.3. `swap_buffers()`

```
void udit::Window::swap_buffers ()
```

Intercambia los buffers de la ventana.

Este método intercambia los buffers del contexto OpenGL, mostrando el contenido renderizado en el buffer de la ventana.

La documentación de esta clase está generada de los siguientes archivos:

- [Code/Headers/Window.hpp](#)
- [Code/Sources/Window.cpp](#)

3.14. Referencia de la clase Window

Clase que gestiona la creación y manipulación de una ventana con contexto OpenGL.

```
#include <Window.hpp>
```

Clases

- struct [OpenGL_Context_Settings](#)
Estructura que contiene los detalles del contexto OpenGL.

Tipos públicos

- enum [Position](#) { [UNDEFINED](#) = SDL_WINDOWPOS_UNDEFINED , [CENTERED](#) = SDL_WINDOWPOS_↔
CENTERED }
- Define las posiciones predefinidas para la ventana.*

Métodos públicos

- [Window](#) (const std::string &title, int left_x, int top_y, unsigned width, unsigned height, const [OpenGL_Context_Settings](#) &context_details)
Constructor de la clase [Window](#).
- [Window](#) (const char *title, int left_x, int top_y, unsigned width, unsigned height, const [OpenGL_Context_Settings](#) &context_details)
Constructor de la clase [Window](#) (versión con título como `const char*`).
- [Window](#) (const [Window](#) &)=delete
Constructor de copia deshabilitado.
- [Window](#) ([Window](#) &&other) noexcept
Constructor de movimiento.
- [~Window](#) ()
Destructor de la clase [Window](#).
- [Window](#) & operator= (const [Window](#) &)=delete
Operador de asignación deshabilitado.
- [Window](#) & operator= ([Window](#) &&other) noexcept
Operador de asignación por movimiento.
- void [swap_buffers](#) ()
Intercambia los buffers de la ventana.

3.14.1. Descripción detallada

Clase que gestiona la creación y manipulación de una ventana con contexto OpenGL.

Esta clase proporciona una interfaz sencilla para crear una ventana que utilice OpenGL como contexto de renderizado, configurando los detalles del contexto de OpenGL y ofreciendo funcionalidades como el intercambio de buffers.

3.14.2. Documentación de las enumeraciones miembro de la clase

3.14.2.1. Position

```
enum udit::Window::Position
```

Define las posiciones predefinidas para la ventana.

Esta enumeración contiene las posibles posiciones que la ventana puede tener en la pantalla.

Valores de enumeraciones

UNDEFINED	La posición de la ventana no está definida.
CENTERED	La ventana se centrará en la pantalla.

3.14.3. Documentación de constructores y destructores

3.14.3.1. Window() [1/4]

```
udit::Window::Window (  
    const std::string & title,  
    int left_x,  
    int top_y,  
    unsigned width,  
    unsigned height,  
    const OpenGL_Context_Settings & context_details) [inline]
```

Constructor de la clase [Window](#).

Este constructor crea una ventana con las especificaciones dadas y configura el contexto OpenGL utilizando los parámetros proporcionados.

Parámetros

<i>title</i>	Título de la ventana.
<i>left_x</i>	Posición en el eje X de la ventana.
<i>top_y</i>	Posición en el eje Y de la ventana.
<i>width</i>	Ancho de la ventana.
<i>height</i>	Alto de la ventana.
<i>context_details</i>	Detalles del contexto OpenGL.

3.14.3.2. Window() [2/4]

```
udit::Window::Window (  
    const char * title,  
    int left_x,  
    int top_y,  
    unsigned width,  
    unsigned height,  
    const OpenGL_Context_Settings & context_details)
```

Constructor de la clase [Window](#) (versión con título como `const char*`).

Este constructor permite crear una ventana con un título de tipo `const char*` y las especificaciones necesarias para el contexto OpenGL.

Parámetros

<i>title</i>	Título de la ventana.
<i>left_x</i>	Posición en el eje X de la ventana.
<i>top_y</i>	Posición en el eje Y de la ventana.
<i>width</i>	Ancho de la ventana.
<i>height</i>	Alto de la ventana.
<i>context_details</i>	Detalles del contexto OpenGL.

3.14.3.3. Window() [3/4]

```
udit::Window::Window (  
    const Window & ) [delete]
```

Constructor de copia deshabilitado.

Este constructor de copia ha sido deshabilitado para evitar la copia del objeto `Window`, ya que no tiene sentido clonar una ventana.

3.14.3.4. Window() [4/4]

```
udit::Window::Window (  
    Window && other) [inline], [noexcept]
```

Constructor de movimiento.

Este constructor transfiere la propiedad de la ventana y el contexto OpenGL de un objeto `Window` a otro mediante el uso de `std::exchange`.

Parámetros

<i>other</i>	Objeto <code>Window</code> cuyo contenido se moverá.
--------------	--

3.14.3.5. ~Window()

```
udit::Window::~~Window ()
```

Destructor de la clase `Window`.

Este destructor destruye la ventana y limpia el contexto OpenGL asociado.

3.14.4. Documentación de funciones miembro

3.14.4.1. operator=() [1/2]

```
Window & udit::Window::operator= (  
    const Window & ) [delete]
```

Operador de asignación deshabilitado.

El operador de asignación ha sido deshabilitado para evitar la asignación de un objeto `Window` a otro, ya que este tipo de operación no es válida en este contexto.

3.14.4.2. operator=() [2/2]

```
Window & udit::Window::operator= (  
    Window && other) [inline], [noexcept]
```

Operador de asignación por movimiento.

Este operador transfiere la propiedad de la ventana y el contexto OpenGL de un objeto `Window` a otro mediante el uso de `std::exchange`.

Parámetros

<i>other</i>	Objeto Window cuyo contenido se moverá.
--------------	---

Devuelve

Referencia al objeto [Window](#) actualizado.

3.14.4.3. swap_buffers()

```
void udit::Window::swap_buffers ()
```

Intercambia los buffers de la ventana.

Este método intercambia los buffers del contexto OpenGL, mostrando el contenido renderizado en el buffer de la ventana.

La documentación de esta clase está generada de los siguientes archivos:

- [Code/Headers/Window.hpp](#)
- [Code/Sources/Window.cpp](#)

Capítulo 4

Documentación de archivos

4.1. Camera.hpp

```
00001 /*@file Camera.hpp
00002  * @author andrmatgonros@gmail.com
00003  * @date 2025-01-12
00004  *
00005  * Este código es de dominio público.
00006  *
00007  * @brief Definición de la clase Camera que permite controlar una cámara 3D.
00008  *
00009  * La clase Camera gestiona la posición y orientación de una cámara en un
00010  * espacio tridimensional. Incluye métodos para mover la cámara mediante teclado
00011  * y rotarla usando el ratón. También proporciona la matriz de vista actualizada
00012  * según las transformaciones aplicadas a la cámara.
00013  */
00014
00015 #pragma once
00016
00017 #include <glm.hpp>
00018 #include <glm/matrix_transform.hpp>
00019 #include <SDL.h>
00020
00021 namespace udit
00022 {
00023     class Camera
00024     {
00025     private:
00026         glm::vec3 position;
00027         glm::vec3 front;
00028         glm::vec3 up;
00029         glm::vec3 right;
00030         glm::vec3 world_up;
00031
00032         float yaw;
00033         float pitch;
00034         float movement_speed;
00035         float mouse_sensitivity;
00036
00037     public:
00038         Camera(
00039             glm::vec3 start_position,
00040             glm::vec3 start_up,
00041             float start_yaw,
00042             float start_pitch
00043         );
00044
00045         glm::mat4 get_view_matrix() const;
00046
00047         void process_keyboard(const Uint8* state, float delta_time);
00048
00049         void start_camera_control();
00050
00051         void process_mouse_motion(float xrel, float yrel);
00052
00053     private:
00054         void update_camera_vectors();
00055     };
00056 }
```

4.2. Cone.hpp

```

00001 /*@file Cone.hpp
00002 * @author andrmatgonros@gmail.com
00003 * @date 2025-01-12
00004 *
00005 * Este código es de dominio público.
00006 *
00007 * @brief Definición de la clase Cone que representa un cono 3D.
00008 *
00009 * La clase Cone se utiliza para generar y renderizar un cono 3D en OpenGL. Permite
00010 * especificar el número de divisiones, el radio de la base y la altura del cono.
00011 * También gestiona los VBOs y VAOs necesarios para la representación del cono.
00012 */
00013
00014 #pragma once
00015 #include <glad/glad.h>
00016 #include <vector>
00017 #include <cmath>
00018 #include <numbers>
00019
00020 using namespace std;
00021
00022 namespace udit
00023 {
00024     class Cone
00025     {
00026     public:
00027         Cone(int d, GLfloat r, GLfloat h);
00028
00029         ~Cone();
00030
00031         void render();
00032
00033     private:
00034         int divisions;
00035         GLfloat radius;
00036         GLfloat height;
00037
00038         vector<GLfloat> coordinates;
00039         vector<GLfloat> texCoords;
00040         vector<GLubyte> indices;
00041
00042         GLuint vao_id;
00043         GLuint vbo_ids[3];
00044
00045         enum { COORDINATES_VBO, INDICES_EBO, TEXCOORDS_VBO, VBO_COUNT };
00046
00047         void generateGeometry();
00048     };
00049 }

```

4.3. Cylinder.hpp

```

00001 /*@file Cylinder.hpp
00002 * @author andrmatgonros@gmail.com
00003 * @date 2025-01-12
00004 *
00005 * Este código es de dominio público.
00006 *
00007 * @brief Definición de la clase Cylinder que representa un cilindro en un espacio 3D.
00008 *
00009 * Este archivo contiene la declaración de la clase Cylinder, que se encarga de la generación
00010 * de la geometría de un cilindro, la configuración de los buffers necesarios para su renderizado
00011 * utilizando OpenGL y su renderización en pantalla.
00012 */
00013
00014 #pragma once
00015 #include <glad/glad.h>
00016 #include <numbers>
00017 #include <cmath>
00018 #include <cstdlib>
00019 #include <vector>
00020
00021 using namespace std;
00022
00023 namespace udit
00024 {
00025     class Cylinder
00026     {
00027     public:
00028         Cylinder(int stack, int slice, GLfloat r, GLfloat h);

```



```

00048
00055     ~Cylinder();
00056
00062     void render();
00063
00064     private:
00065         int stack_count;
00066         int slice_count;
00067         GLfloat radius;
00068         GLfloat height;
00069
00070         vector<GLfloat> coordinates;
00071         vector<GLfloat> texCoords;
00072         vector<GLubyte> indices;
00073
00074         GLuint vao_id;
00075         GLuint vbo_ids[4];
00076
00077         enum { COORDINATES_VBO, TEXCOORDS_VBO, INDICES_EBO, VBO_COUNT };
00078
00086         void generateGeometry();
00087     };
00088 }

```

4.4. Heightmap.hpp

```

00001 /*@file Heightmap.hpp
00002  * @author andrmatgonros@gmail.com
00003  * @date 2025-01-12
00004  *
00005  * Este código es de dominio público.
00006  *
00007  * @brief Definición de la clase Heightmap para manejar mapas de altura en la generación de terrenos.
00008  *
00009  * Esta clase es responsable de cargar un mapa de altura desde una imagen, generar la malla de
00010  * vértices
00011  * que representa el terreno y renderizarla utilizando OpenGL. El mapa de altura es una imagen de
00012  * escala
00013  * de grises donde cada píxel define la elevación del terreno en un punto específico.
00014  */
00015 #pragma once
00016 #include <vector>
00017 #include <string>
00018 #include <glm.hpp>
00019 #include <glad/glad.h>
00020 namespace udit
00021 {
00022     class Heightmap {
00023     public:
00024         Heightmap(const std::string& heightmapPath, float width, float height, float maxHeight);
00025         ~Heightmap();
00026         void render() const;
00027         GLuint getTextureID() const { return textureID; }
00028     private:
00029         GLuint vao;
00030         GLuint vbo;
00031         GLuint ebo;
00032         GLuint textureID;
00033         unsigned int numIndices;
00034         void generateMesh(const std::vector<float>& heightData, int width, int height, float
00035         maxHeight);
00036         std::vector<float> loadHeightData(const std::string& heightmapPath, int& width, int&
00037         height);
00038     };
00039 }

```

4.5. Plane.hpp

```

00001 /*@file Plane.hpp

```

```

00002  * @author andrmatgonros@gmail.com
00003  * @date 2025-01-12
00004  *
00005  * Este código es de dominio público.
00006  *
00007  * @brief Declaración de la clase Plane para representar un plano 3D.
00008  *
00009  * Esta clase genera un plano de malla utilizando OpenGL, donde el plano tiene una
00010  * cuadrícula definida por su ancho y alto. Se utiliza para representar superficies planas.
00011  */
00012
00013 #pragma once
00014 #include <glad/glad.h>
00015 #include <vector>
00016 using namespace std;
00017
00018 namespace udit
00019 {
00020     class Plane
00021     {
00022     public:
00023         Plane(int width, int height);
00024
00025         ~Plane();
00026
00027         void render();
00028
00029     private:
00030         GLuint vao_id;
00031         GLuint vbo_ids[3];
00032
00033         int grid_width;
00034         int grid_height;
00035
00036         enum { COORDINATES_VBO, TEXCOORDS_VBO, INDICES_EBO, VBO_COUNT };
00037
00038         vector<GLfloat> coordinates;
00039         vector<GLfloat> texCoords;
00040         vector<GLubyte> indices;
00041
00042         void generateGeometry();
00043     };
00044 }

```

4.6. Referencia del archivo Code/Headers/Scene.hpp

Definición de la clase [Scene](#), encargada de manejar y renderizar la escena 3D.

```

#include "Cylinder.hpp"
#include "Heightmap.hpp"
#include "Cone.hpp"
#include "Plane.hpp"
#include "Camera.hpp"
#include "TextureLoader.hpp"
#include "Skybox.hpp"
#include "Sphere.hpp"
#include <string>
#include <iostream>
#include <cassert>
#include <SDL.h>
#include <glm.hpp>
#include <gtc/matrix_transform.hpp>
#include <gtc/type_ptr.hpp>

```

Clases

- [class udit::Scene](#)

Clase para representar y gestionar una escena 3D.

4.6.1. Descripción detallada

Definición de la clase [Scene](#), encargada de manejar y renderizar la escena 3D.

Autor

Original Author angel.rodriguez@udit.es
andrmartgonros@gmail.com

Fecha

2025-01-12

Este código es de dominio público.

Esta clase es responsable de gestionar los objetos 3D en la escena, las texturas, las cámaras, y los shaders. También se encarga de las operaciones de actualización y renderizado de la escena utilizando OpenGL.

Nota

Modificado por andrmartgonros@gmail.com para extender la funcionalidad original de la clase [Scene](#). La versión original sólo renderizaba un cubo girando. Se agregaron los siguientes elementos:

- Skybox: Representación de un entorno 3D que rodea la escena.
- Cámara: Implementación de una cámara interactiva para moverse por la escena.
- Nuevos objetos 3D: Se añadieron un plano, un cilindro, un cono, una esfera y un heightmap.
- TextureLoader: Creación de un cargador de texturas personalizado para mejorar la carga de texturas en la escena.

4.7. Scene.hpp

[Ir a la documentación de este archivo.](#)

```
00001
00021
00022 #pragma once
00023
00024 #include "Cylinder.hpp"
00025 #include "Heightmap.hpp"
00026 #include "Cone.hpp"
00027 #include "Plane.hpp"
00028 #include "Camera.hpp"
00029 #include "TextureLoader.hpp"
00030 #include "Skybox.hpp"
00031 #include "Sphere.hpp"
00032 #include <string>
00033 #include <iostream>
00034 #include <cassert>
00035 #include <SDL.h>
00036 #include <glm.hpp>
00037 #include <gtc/matrix_transform.hpp>
00038 #include <gtc/type_ptr.hpp>
00039
00040 namespace udit
00041 {
00056     class Scene
00057     {
00058     private:
00059         // Códigos de los shaders
00060         static const std::string vertex_shader_code;
00061         static const std::string fragment_shader_code;
00062         static const std::string skybox_vertex_shader;
00063         static const std::string skybox_fragment_shader;
00064         static const std::string heightmap_vertex_shader;
```

```

00065         static const std::string heightmap_fragment_shader;
00066
00067         // Identificadores de las matrices y programas de los shaders
00068         GLint model_view_matrix_id;
00069         GLint projection_matrix_id;
00070         GLuint program_id;
00071         GLuint skybox_program_id;
00072
00073         // Objetos 3D de la escena
00074         Skybox skybox;
00075         Sphere sphere1;
00076         Sphere sphere2;
00077         Cone cone;
00078         Cylinder cylinder;
00079         Plane plane;
00080         Heightmap heightmap;
00081
00082         // Ángulo de rotación de la escena.
00083         float angle;
00084
00085         // Cámara que gestiona la vista.
00086         Camera camera;
00087
00088         // Gestor de texturas
00089         TextureLoader textureLoader;
00090
00091         // Identificadores de las texturas para cada objeto 3D
00092         GLuint cubeTextureID;
00093         GLuint planeTextureID;
00094         GLuint cylinderTextureID;
00095         GLuint coneTextureID;
00096         GLuint skyboxTextureID;
00097         GLuint sphereTextureID;
00098         GLuint heightmapTextureID;
00099
00100         GLuint compile_shaders();
00101
00102         GLuint compile_skybox_shaders();
00103
00104         void show_compilation_error(GLuint shader_id);
00105
00106         void show_linkage_error(GLuint program_id);
00107
00108     public:
00109         Scene(unsigned width, unsigned height);
00110
00111         void update(float delta_time);
00112
00113         void render();
00114
00115         void resize(unsigned width, unsigned height);
00116
00117         void handle_mouse_motion(float xrel, float yrel);
00118     };
00119 }
00120

```

4.8. Referencia del archivo Code/Headers/Skybox.hpp

```

#include <string>
#include <vector>
#include <glad/glad.h>
#include <gtc/type_ptr.hpp>
#include <glm.hpp>
#include <iostream>
#include "../Headers/stb_image.hpp"

```

Clases

- class `udit::Skybox`

Clase que representa un skybox en una escena 3D.

4.8.1. Descripción detallada

Autor

andrmatorgonros@gmail.com

Fecha

2025-01-12

Este archivo contiene la declaración de la clase Skybox, que es responsable de manejar un skybox en una escena 3D. Un skybox es una estructura que envuelve toda la escena, normalmente usada para representar el fondo del entorno, como el cielo, usando texturas de cubo. Esta clase permite cargar las texturas del skybox, crear buffers de OpenGL y renderizar el skybox.

4.9. Skybox.hpp

[Ir a la documentación de este archivo.](#)

```
00001
00011
00012 #pragma once
00013
00014 #include <string>
00015 #include <vector>
00016 #include <glad/glad.h>
00017 #include <gtc/type_ptr.hpp>
00018 #include <glm.hpp>
00019 #include <iostream>
00020 #include "../Headers/stb_image.hpp"
00021 namespace udit
00022 {
00023
00024     class Skybox {
00025     public:
00026         Skybox(const std::vector<std::string>& faces);
00027
00028         ~Skybox();
00029
00030         void set_texture(GLuint texture_id);
00031
00032         GLuint get_texture_id();
00033
00034         void render();
00035
00036     private:
00037         GLuint vao_id;
00038         GLuint vbo_id;
00039         GLuint texture_id;
00040         std::vector<std::string> faces;
00041
00042         void setup_buffers();
00043     };
00044 }
```

4.10. Referencia del archivo Code/Headers/Sphere.hpp

```
#include <glad/glad.h>
#include <vector>
#include <cmath>
#include <numbers>
```

Clases

- class `udit::Sphere`

Clase que representa una esfera 3D generada por subdivisiones esféricas.

4.10.1. Descripción detallada

Autor

`andrmaggonros@gmail.com`

Fecha

2025-01-12

Este archivo define la clase `Sphere`, que se utiliza para generar y renderizar una esfera 3D en OpenGL. La esfera se define a partir de divisiones latitudinales y longitudinales, lo que permite controlar la resolución de la geometría de la esfera.

4.11. Sphere.hpp

[Ir a la documentación de este archivo.](#)

```
00001
00010
00011 #pragma once
00012 #include <glad/glad.h>
00013 #include <vector>
00014 #include <cmath>
00015 #include <numbers>
00016
00017 using namespace std;
00018
00019 namespace udit
00020 {
00030     class Sphere
00031     {
00032     public:
00043         Sphere(int latDivisions, int longDivisions, GLfloat radius);
00044
00051         ~Sphere();
00052
00059         void render();
00060
00061     private:
00062         int latitudeDivisions;
00063         int longitudeDivisions;
00064         GLfloat radius;
00065
00066         vector<GLfloat> coordinates;
00067         vector<GLfloat> texCoords;
00068         vector<GLubyte> indices;
00069
00070         GLuint vao_id;
00071         GLuint vbo_ids[3];
00072
00073         enum { COORDINATES_VBO, INDICES_EBO, TEXCOORDS_VBO, VBO_COUNT };
00074
00082         void generateGeometry();
00083     };
00084 }
```

4.12. Referencia del archivo Code/Headers/TextureLoader.hpp

```
#include <string>
#include <vector>
#include <glad/glad.h>
#include <iostream>
#include "../Headers/stb_image.hpp"
```

Clases

- `class udit::TextureLoader`

Clase para cargar texturas en OpenGL.

4.12.1. Descripción detallada

Autor

`andrmaggonros@gmail.com`

Fecha

2025-01-12

Este archivo contiene la declaración de la clase `TextureLoader`, que facilita la carga de texturas en OpenGL, incluyendo tanto texturas 2D normales como cubemaps para efectos de entorno.

4.13. TextureLoader.hpp

[Ir a la documentación de este archivo.](#)

```
00001
00010
00011 #pragma once
00012
00013 #include <string>
00014 #include <vector>
00015 #include <glad/glad.h>
00016 #include <iostream>
00017 #include "../Headers/stb_image.hpp"
00018
00019 using namespace std;
00020 namespace udit
00021 {
00022
00030     class TextureLoader {
00031     public:
00038         TextureLoader();
00039
00045         ~TextureLoader();
00046
00055         GLuint loadTexture(const string& filePath);
00056
00065         GLuint loadCubemap(const vector<string>& faces);
00066
00067     private:
00068         GLuint textureID;
00069     };
00070 }
```

4.14. Referencia del archivo Code/Headers/Window.hpp

```
#include <SDL.h>
#include <string>
#include <utility>
```

Clases

- class [udit::Window](#)

Clase que gestiona la creación y manipulación de una ventana con contexto OpenGL.

- struct [udit::Window::OpenGL_Context_Settings](#)

Estructura que contiene los detalles del contexto OpenGL.

4.14.1. Descripción detallada

Autor

angel.rodriguez@udit.es

Fecha

2025-01-12

Este archivo contiene la definición de la clase [Window](#), que facilita la creación y manejo de una ventana con contexto OpenGL utilizando SDL.

4.15. Window.hpp

[Ir a la documentación de este archivo.](#)

```
00001
00009
00010 #pragma once
00011
00012 #include <SDL.h>
00013 #include <string>
00014 #include <utility>
00015
00016 namespace udit
00017 {
00018
00027     class Window
00028     {
00029     public:
00030
00037         enum Position
00038         {
00039             UNDEFINED = SDL_WINDOWPOS_UNDEFINED,
00040             CENTERED = SDL_WINDOWPOS_CENTERED,
00041         };
00042
00049         struct OpenGL_Context_Settings
00050         {
00051             unsigned version_major = 3;
00052             unsigned version_minor = 3;
00053             bool core_profile = true;
00054             unsigned depth_buffer_size = 24;
00055             unsigned stencil_buffer_size = 0;
00056             bool enable_vsync = true;
00057         };
00057     };
00057 }
```



```

00058
00059     private:
00060
00061         SDL_Window* window_handle;
00062         SDL_GLContext opengl_context;
00063
00064     public:
00065
00079         Window
00080         (
00081             const std::string& title,
00082             int left_x,
00083             int top_y,
00084             unsigned width,
00085             unsigned height,
00086             const OpenGL_Context_Settings& context_details
00087         )
00088         :
00089         Window(title.c_str(), left_x, top_y, width, height, context_details)
00090         {
00091         }
00092
00106         Window
00107         (
00108             const char* title,
00109             int left_x,
00110             int top_y,
00111             unsigned width,
00112             unsigned height,
00113             const OpenGL_Context_Settings& context_details
00114         );
00115
00121         ~Window();
00122
00123     public:
00124
00131         Window(const Window&) = delete;
00132
00139         Window& operator = (const Window&) = delete;
00140
00149         Window(Window&& other) noexcept
00150         {
00151             this->window_handle = std::exchange(other.window_handle, nullptr);
00152             this->opengl_context = std::exchange(other.opengl_context, nullptr);
00153         }
00154
00164         Window& operator = (Window&& other) noexcept
00165         {
00166             this->window_handle = std::exchange(other.window_handle, nullptr);
00167             this->opengl_context = std::exchange(other.opengl_context, nullptr);
00168         }
00169
00170     public:
00171
00178         void swap_buffers();
00179
00180 };
00181
00182 }
```


Índice alfabético

- ~Cone
 - udit::Cone, [8](#)
- ~Cylinder
 - udit::Cylinder, [9](#)
- ~Heightmap
 - udit::Heightmap, [10](#)
- ~Plane
 - udit::Plane, [15](#)
- ~Skybox
 - udit::Skybox, [20](#)
- ~Sphere
 - udit::Sphere, [22](#)
- ~TextureLoader
 - udit::TextureLoader, [23](#)
- ~Window
 - udit::Window, [26](#)
 - Window, [31](#)
- Camera
 - udit::Camera, [5](#)
- CENTERED
 - udit::Window, [26](#)
 - Window, [29](#)
- Code/Headers/Camera.hpp, [33](#)
- Code/Headers/Cone.hpp, [34](#)
- Code/Headers/Cylinder.hpp, [34](#)
- Code/Headers/Heightmap.hpp, [35](#)
- Code/Headers/Plane.hpp, [35](#)
- Code/Headers/Scene.hpp, [36, 37](#)
- Code/Headers/Skybox.hpp, [38, 39](#)
- Code/Headers/Sphere.hpp, [39, 40](#)
- Code/Headers/TextureLoader.hpp, [41](#)
- Code/Headers/Window.hpp, [42](#)
- Cone
 - udit::Cone, [7](#)
- core_profile
 - udit::Window::OpenGL_Context_Settings, [12](#)
 - Window::OpenGL_Context_Settings, [13](#)
- Cylinder
 - udit::Cylinder, [9](#)
- depth_buffer_size
 - udit::Window::OpenGL_Context_Settings, [12](#)
 - Window::OpenGL_Context_Settings, [13](#)
- enable_vsync
 - udit::Window::OpenGL_Context_Settings, [12](#)
 - Window::OpenGL_Context_Settings, [13](#)
- get_texture_id
 - udit::Skybox, [21](#)
- get_view_matrix
 - udit::Camera, [6](#)
- getTextureID
 - udit::Heightmap, [11](#)
- handle_mouse_motion
 - Scene, [16](#)
 - udit::Scene, [18](#)
- Heightmap
 - udit::Heightmap, [10](#)
- loadCubemap
 - udit::TextureLoader, [24](#)
- loadTexture
 - udit::TextureLoader, [24](#)
- operator=
 - udit::Window, [27](#)
 - Window, [31](#)
- Plane
 - udit::Plane, [14](#)
- Position
 - udit::Window, [25](#)
 - Window, [29](#)
- process_keyboard
 - udit::Camera, [6](#)
- process_mouse_motion
 - udit::Camera, [6](#)
- render
 - Scene, [16](#)
 - udit::Cone, [8](#)
 - udit::Cylinder, [9](#)
 - udit::Heightmap, [11](#)
 - udit::Plane, [15](#)
 - udit::Scene, [19](#)
 - udit::Skybox, [21](#)
 - udit::Sphere, [23](#)
- resize
 - Scene, [17](#)
 - udit::Scene, [19](#)
- Scene, [15](#)
 - handle_mouse_motion, [16](#)
 - render, [16](#)
 - resize, [17](#)
 - Scene, [16](#)
 - udit::Scene, [18](#)

- update, 17
- set_texture
 - udit::Skybox, 21
- Skybox
 - udit::Skybox, 20
- Sphere
 - udit::Sphere, 22
- start_camera_control
 - udit::Camera, 7
- stencil_buffer_size
 - udit::Window::OpenGL_Context_Settings, 12
 - Window::OpenGL_Context_Settings, 13
- swap_buffers
 - udit::Window, 28
 - Window, 32
- TextureLoader
 - udit::TextureLoader, 23
- udit::Camera, 5
 - Camera, 5
 - get_view_matrix, 6
 - process_keyboard, 6
 - process_mouse_motion, 6
 - start_camera_control, 7
- udit::Cone, 7
 - ~Cone, 8
 - Cone, 7
 - render, 8
- udit::Cylinder, 8
 - ~Cylinder, 9
 - Cylinder, 9
 - render, 9
- udit::Heightmap, 10
 - ~Heightmap, 10
 - getTextureID, 11
 - Heightmap, 10
 - render, 11
- udit::Plane, 14
 - ~Plane, 15
 - Plane, 14
 - render, 15
- udit::Scene, 17
 - handle_mouse_motion, 18
 - render, 19
 - resize, 19
 - Scene, 18
 - update, 19
- udit::Skybox, 20
 - ~Skybox, 20
 - get_texture_id, 21
 - render, 21
 - set_texture, 21
 - Skybox, 20
- udit::Sphere, 22
 - ~Sphere, 22
 - render, 23
 - Sphere, 22
- udit::TextureLoader, 23
 - ~TextureLoader, 23
 - loadCubemap, 24
 - loadTexture, 24
 - TextureLoader, 23
- udit::Window, 25
 - ~Window, 26
 - CENTERED, 26
 - operator=, 27
 - Position, 25
 - swap_buffers, 28
 - UNDEFINED, 26
 - Window, 26, 27
- udit::Window::OpenGL_Context_Settings, 11
 - core_profile, 12
 - depth_buffer_size, 12
 - enable_vsync, 12
 - stencil_buffer_size, 12
 - version_major, 12
 - version_minor, 12
- UNDEFINED
 - udit::Window, 26
 - Window, 29
- update
 - Scene, 17
 - udit::Scene, 19
- version_major
 - udit::Window::OpenGL_Context_Settings, 12
 - Window::OpenGL_Context_Settings, 13
- version_minor
 - udit::Window::OpenGL_Context_Settings, 12
 - Window::OpenGL_Context_Settings, 14
- Window, 28
 - ~Window, 31
 - CENTERED, 29
 - operator=, 31
 - Position, 29
 - swap_buffers, 32
 - udit::Window, 26, 27
 - UNDEFINED, 29
 - Window, 30, 31
- Window::OpenGL_Context_Settings, 13
 - core_profile, 13
 - depth_buffer_size, 13
 - enable_vsync, 13
 - stencil_buffer_size, 13
 - version_major, 13
 - version_minor, 14